

Apellidos, Nombre: Flores Flores, Daniel Javier
Apellidos, Nombre: Lillo Ramírez, Lucas
Apellidos, Nombre:

La entrega definitiva de este proyecto será el 27 de marzo. En la entrega, subid:

- **Este fichero (doc/pdf) con las respuestas/código e imágenes pedidas.**
- **Las dos fotografías y el script fusion.m para obtener la fotografía descrita en el apartado 3.2. Vuestro script debe contener todas las rutinas que hayáis usado para que yo pueda ejecutarlo.**

Para la clase del 20 de abril debéis llevar hecho hasta el punto indicado. En esa clase revisaré/puntuaré lo que tengáis hecho, resolveremos dudas, y seguiréis trabajando con el resto de los apartados, ...

1: Uso de filtros al cambiar el tamaño de una imagen

Reducción del tamaño de una imagen:

Adjuntad código de la función y los filtros obtenidos para N=3 y N=5.

Código función H

```
function H = raised_cos(N)
    L = (N-1)/2;
    k = -L:L;
    h = 1 + cos(pi*k/(L+1));
    h = h / sum(h);
    H = h' * h;
end
```

Código función H

```
function im2 = reduce(im)
    im2 = im(1:2:end, 1:2:end, :);
end
```

Filtros obtenidos

H3 =

```
0.0625  0.1250  0.0625
0.1250  0.2500  0.1250
0.0625  0.1250  0.0625
```

H5 =

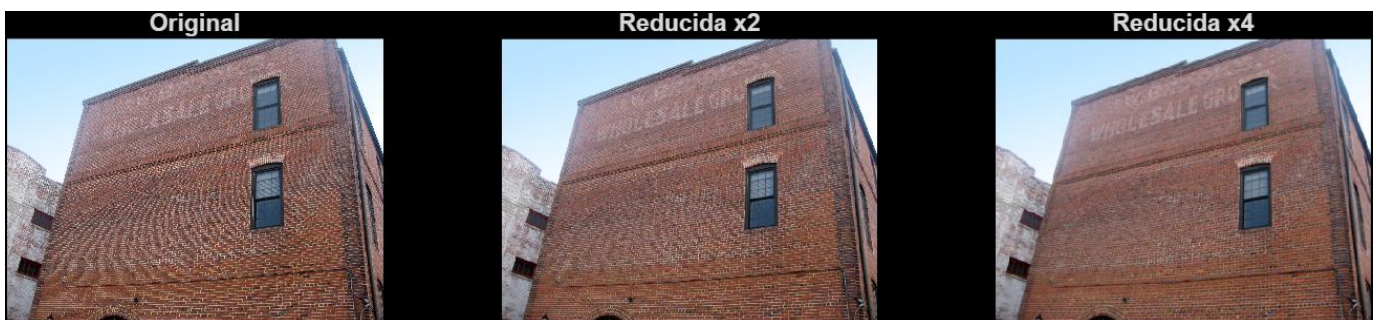
```
0.0069  0.0208  0.0278  0.0208  0.0069
```

0.0208	0.0625	0.0833	0.0625	0.0208
0.0278	0.0833	0.1111	0.0833	0.0278
0.0208	0.0625	0.0833	0.0625	0.0208
0.0069	0.0208	0.0278	0.0208	0.0069

Adjuntad código de la función modificada.

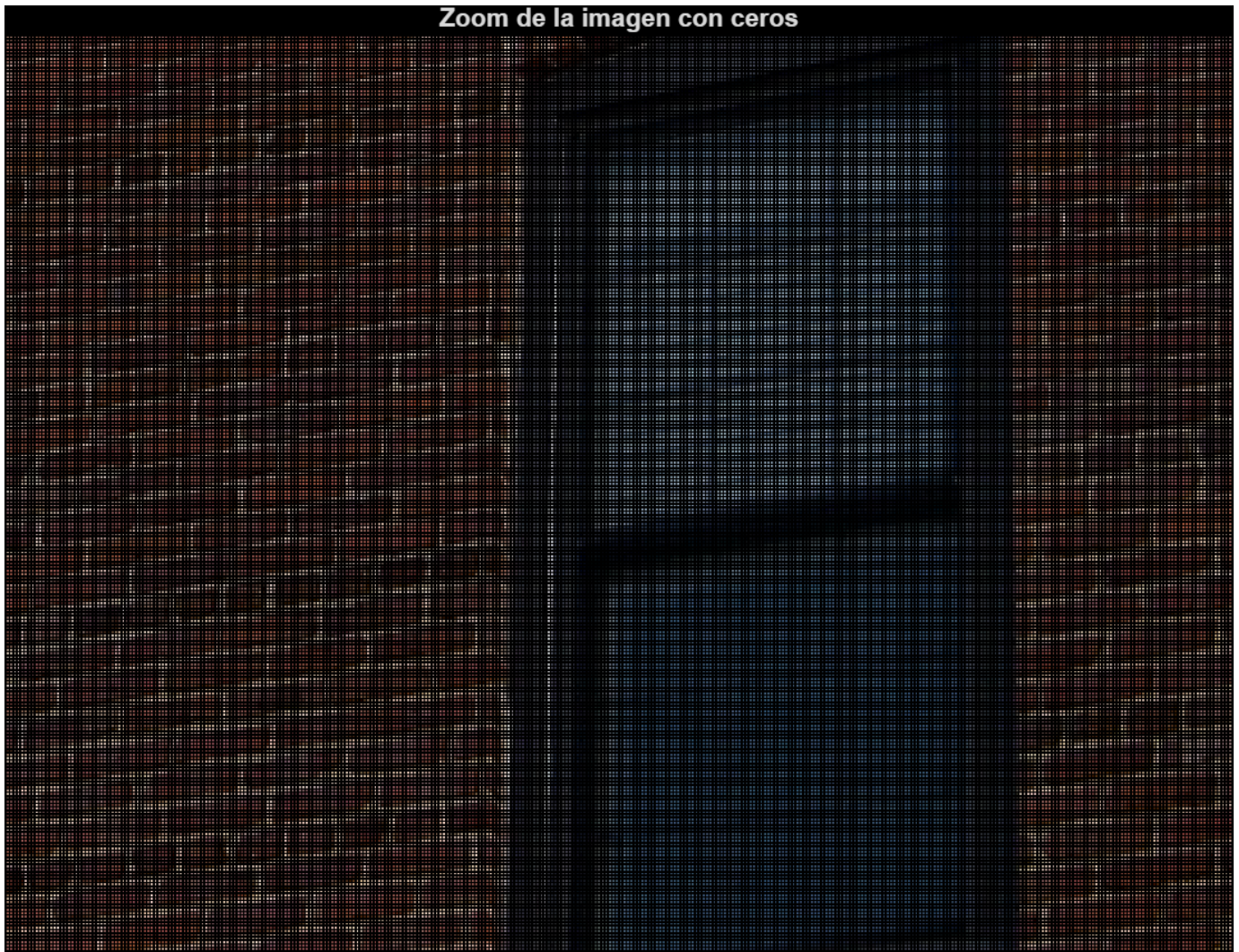
```
function im2 = reduce(im)
    H = raised_cos(3);
    imf = imfilter(im, H, 'same', 'replicate');
    im2 = imf(1:2:end, 1:2:end, :);
end
```

Adjuntad una captura.



Ampliación de una imagen.

Adjuntad una captura del zoom.



Adjuntad la máscara 3x3 que cumple las condiciones pedidas.

Mascara 3x3:

0	0.2500	0
0.2500	1.0000	0.2500
0	0.2500	0

¿Cuál es la suma de sus coeficientes? Justificad este resultado.

La suma es 2 porque al insertar ceros la imagen pierde intensidad. El filtro compensa esta pérdida duplicando la energía para reconstruir correctamente la imagen.

¿Con cuál de los filtros promedio que hemos visto está relacionada esta máscara?

Se relaciona con un filtro gaussiano.

Adjuntad el código de la función ampliar() una vez completada. Adjuntad zoom de la imagen obtenida en la esquina inferior derecha

```
function im2 = amplia(im)
im = im2double(im);
[f, c, ch] = size(im);
```



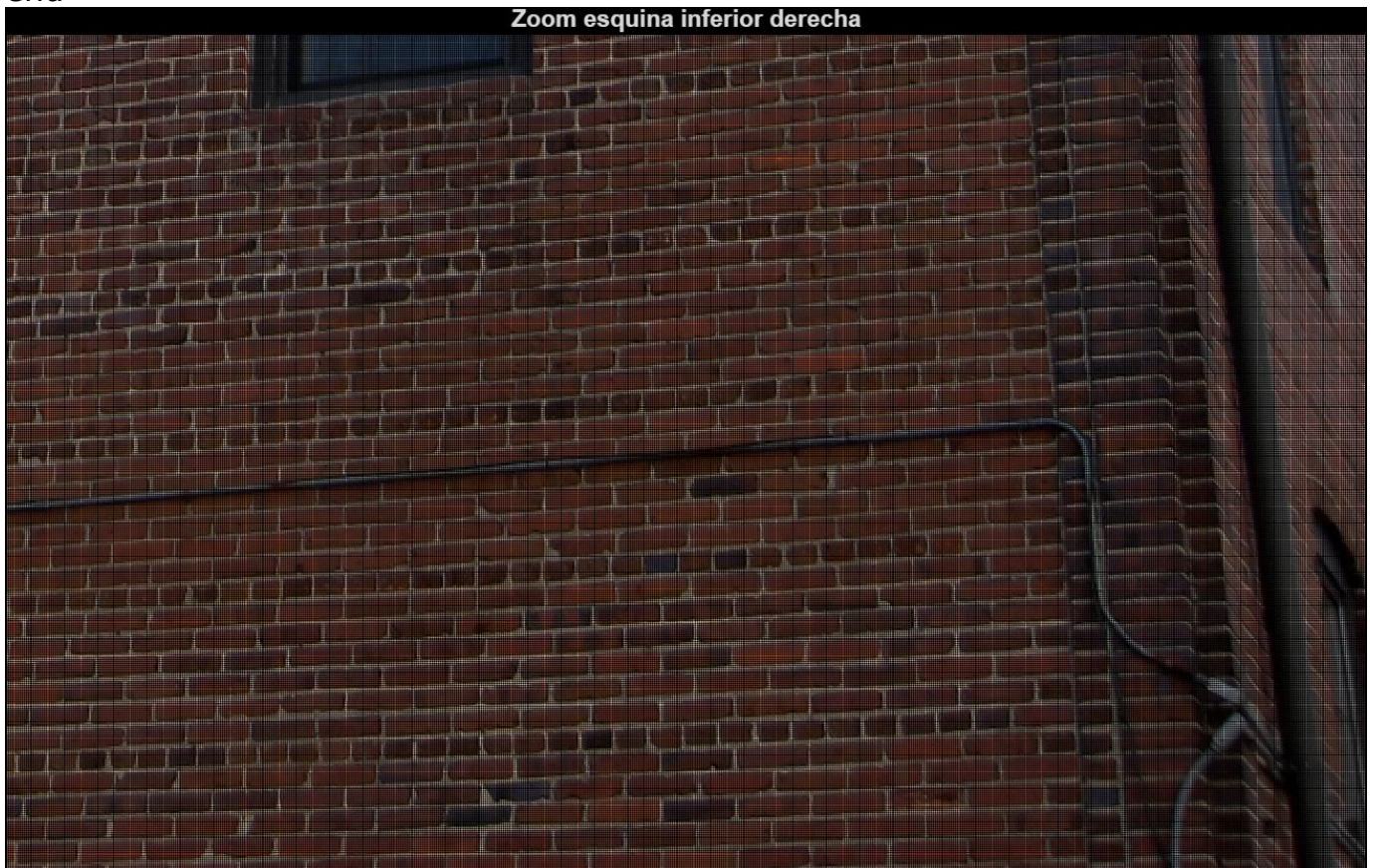
```

im2 = zeros(2*f, 2*c, ch);
im2(1:2:end, 1:2:end, :) = im;

H = [0 1 0;
     1 4 1;
     0 1 0] / 4;

for k = 1:ch
    im2(:, :, k) = imfilter(im2(:, :, k), H, 'replicate');
end
end

```



Dad la máscara 3x3 que podríamos usar para recuperar el canal verde. ¿Cuánto suman ahora sus coeficientes? Justificar.

Mascara Bayer:

0	0.2500	0
0.2500	1.0000	0.2500
0	0.2500	0

La suma de los coeficientes es 2 porque se está interpolando a partir de valores conocidos, y es necesario compensar la falta de información en los píxeles desconocidos.

Realce de bordes:

Volcad el valor mínimo/máximo del detalle de la imagen.

Min detalle:

-0.3102

Max detalle:

0.4516

Adjuntad imagen del detalle visualizado de esta forma.



Dad el valor máximo y mínimo de la imagen obtenida.

Min imagen final:

0

Max imagen final:

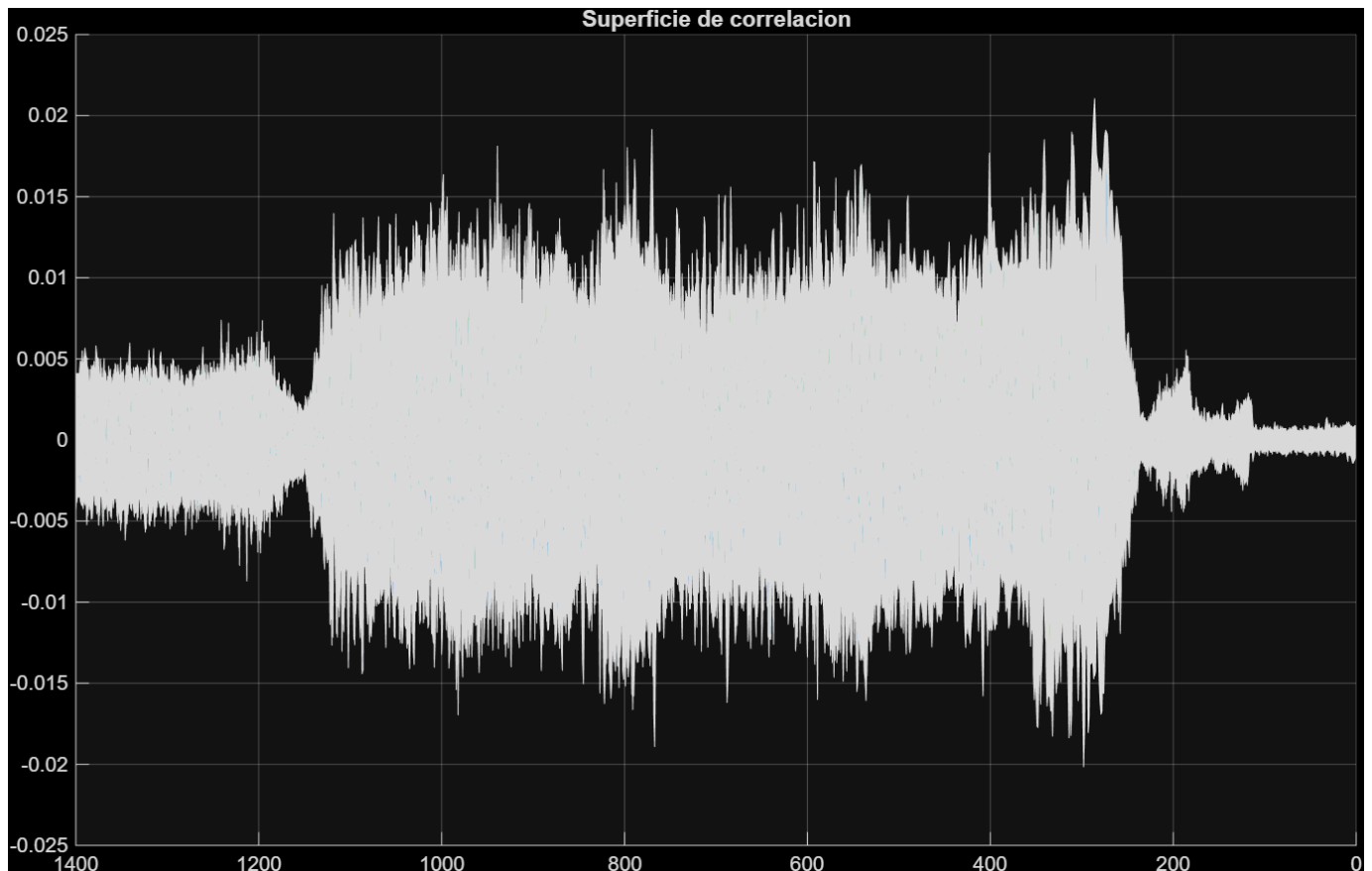
1

Adjuntad la imagen resultado.

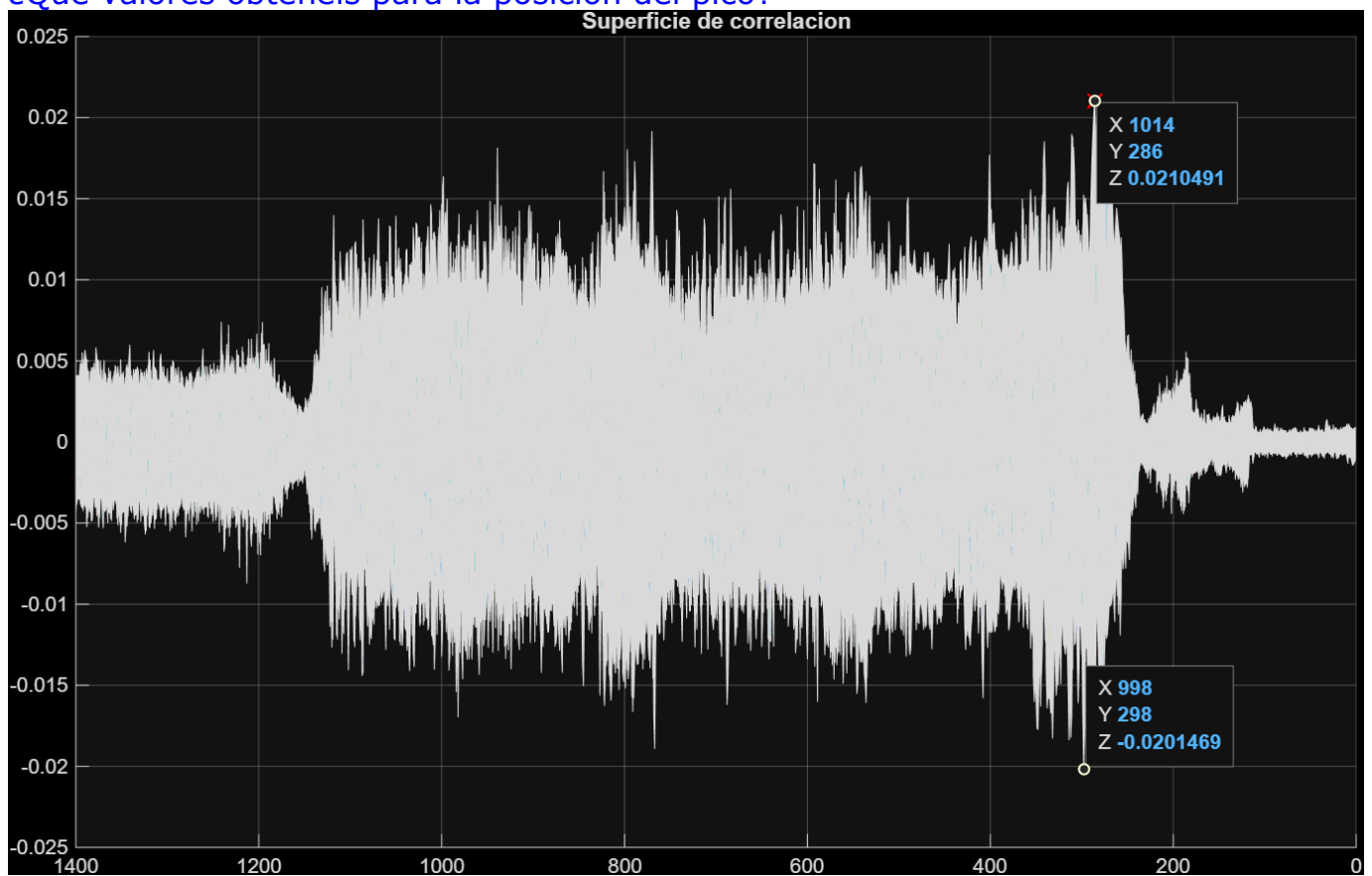


2. Registro de imágenes

[Adjuntad la imagen.](#)



¿Qué valores obtenéis para la posición del pico?



El valor maximo pico es 1014, 286

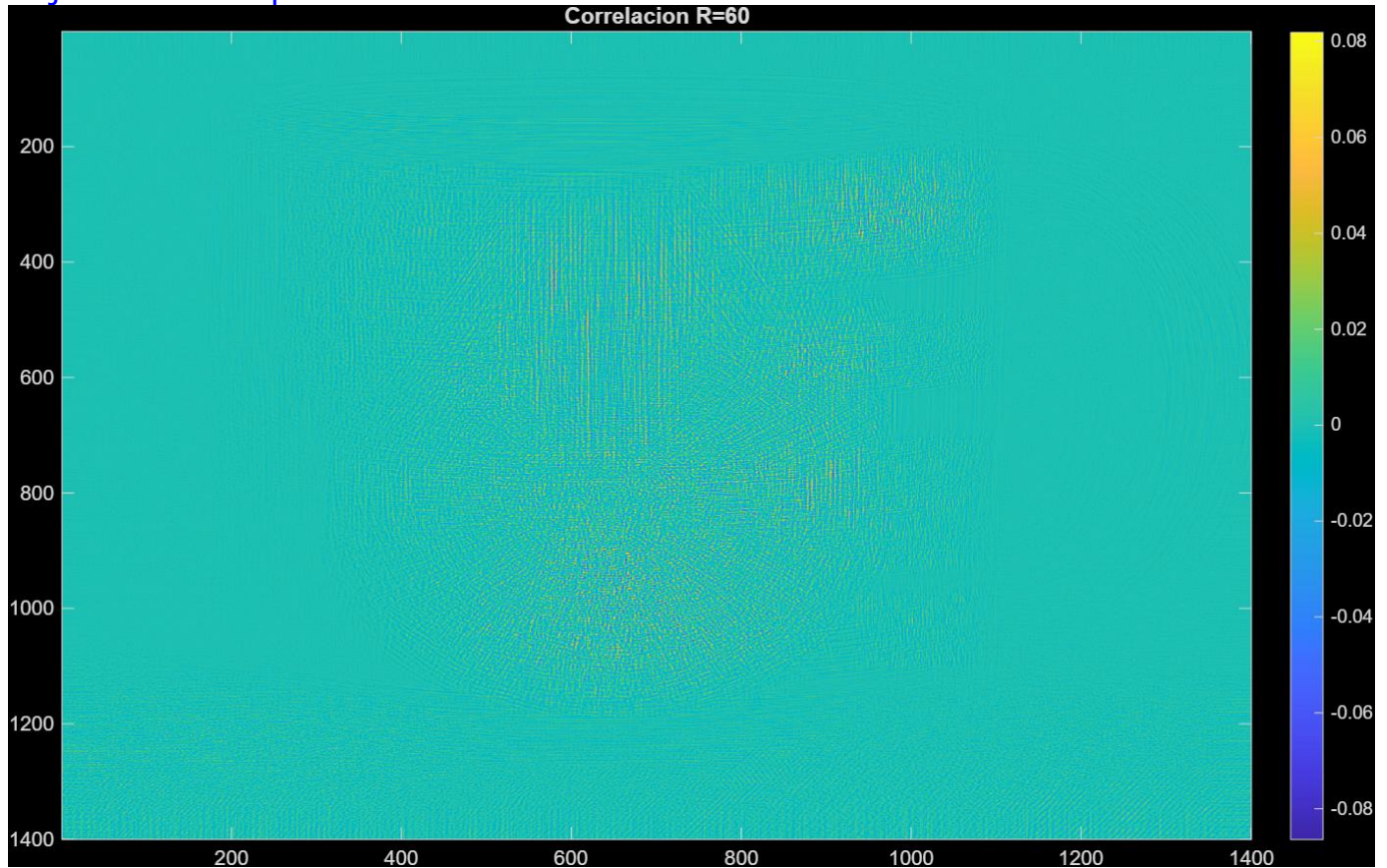
¿Valores obtenidos para dX, dY?

Desplazamiento:

$dx = 969$

$dy = 221$

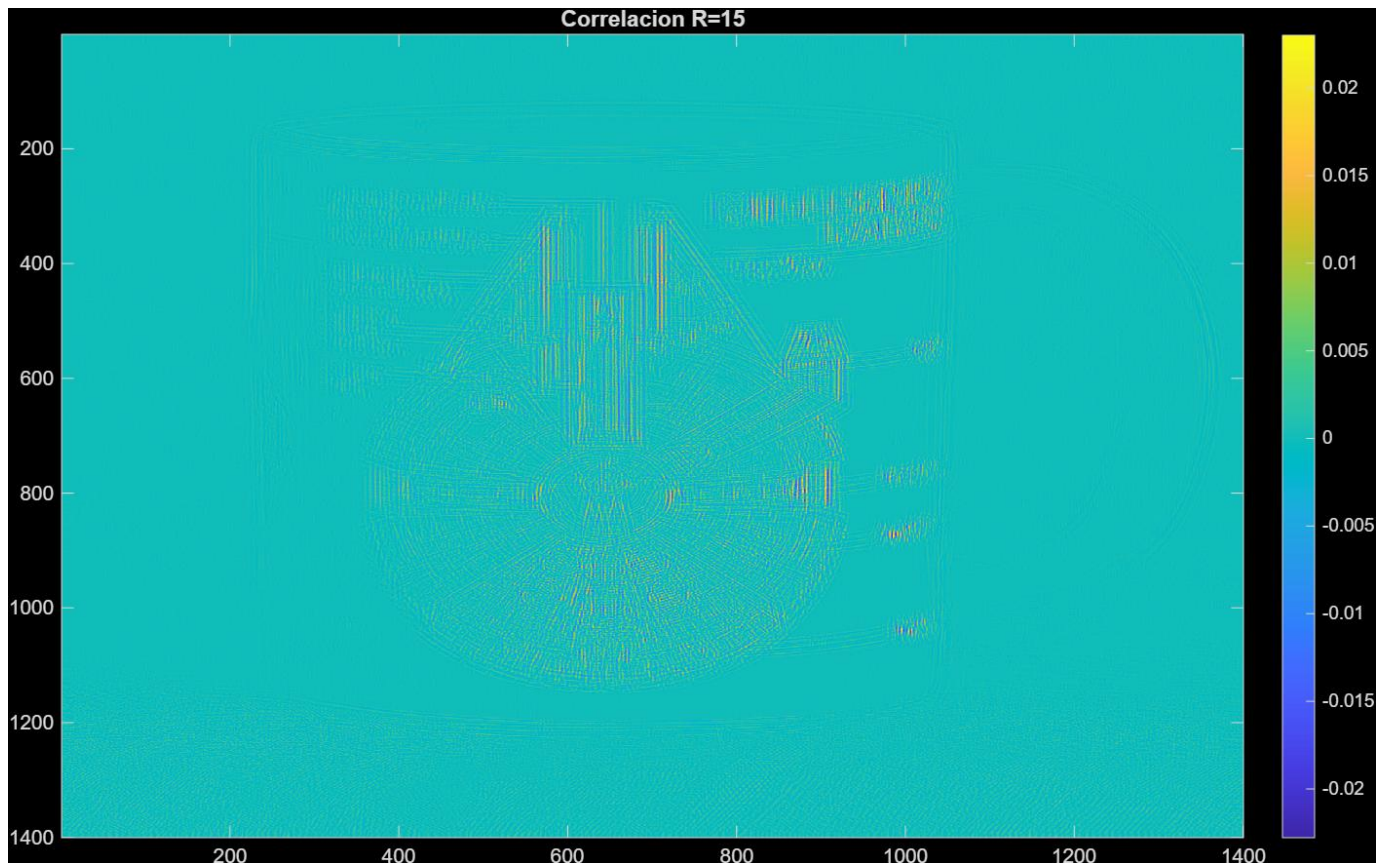
Adjuntad una captura de la nueva correlación obtenida.



¿Por qué creéis que sucede esto?

Al usar una porción de la imagen más grande hay mas información y por lo tanto más elementos y patrones iguales

Adjuntad captura de la correlación obtenida.



¿Obtenéis ahora el resultado correcto para los desplazamientos (dX, dY)?

R=15

871 760

¿Cuál es el problema?

No se obtiene el resultado correcto porque al usar una porción de imagen pequeña hay muy pocos datos para realizar la correlación

3. Pirámide Laplaciana y aplicaciones

3.1 Implementación de la pirámide Laplaciana

Adjuntar código de vuestra función lap().

```
function p = lap(im, N)
```

```
    im = im2double(im);
```

```
    p = cell(1, N);
```

```
    for k = 1:(N-1)
```

```
        im_red = reduce(im);
```



```
im_amp = amplia(im_red);  
  
p{k} = im - im_amp;  
  
im = im_red;  
end  
  
% Guardado  
p{N} = im;  
  
end
```

Adjuntad la imagen resultante.



Inversa de la pirámide Laplaciana: recuperación de la imagen original

Adjuntar el código de vuestra función `inv_lap()`

```
function im = inv_lap(p)
```

```
N = length(p);  
im = p{N};
```



```
for k = (N-1):-1:1

    im = amplia(im);

    % Ajustar tamaño
    f = min(size(im,1), size(p{k},1));
    c = min(size(im,2), size(p{k},2));

    im = im(1:f, 1:c, :);

    im = im + p{k};
end

end
```

Adjuntad vuestros resultados.

Max diferencia:
2.2204e-16

Min diferencia:
0

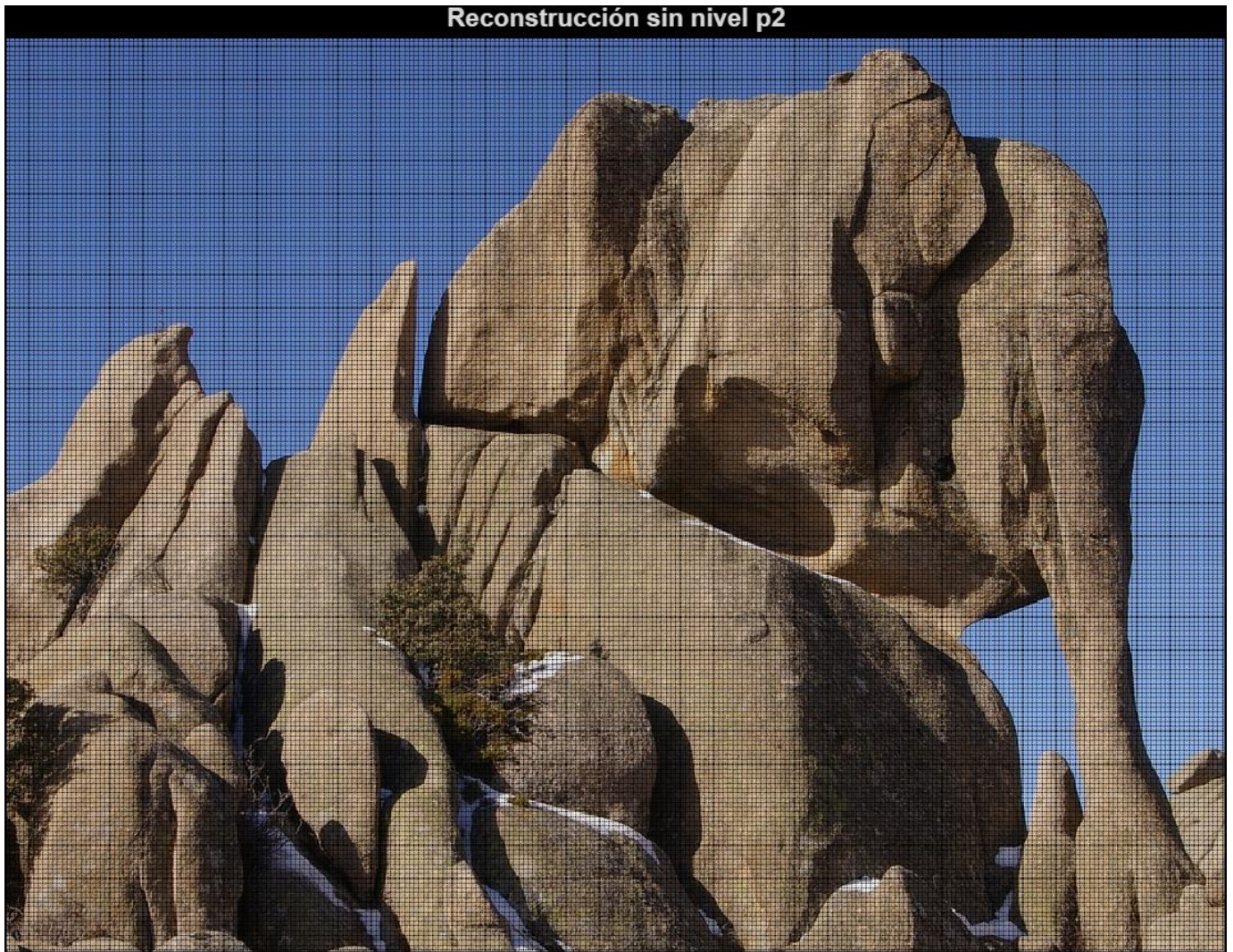
Adjuntad código usado y captura del resultado.

```
%Reconstrucción sin nivel p{2}

p2 = p;
p2{2} = zeros(size(p{2}));

im_rec2 = inv_lap(p2);

figure;
imshow(im_rec2);
title('Reconstrucción sin nivel p{2}');
```



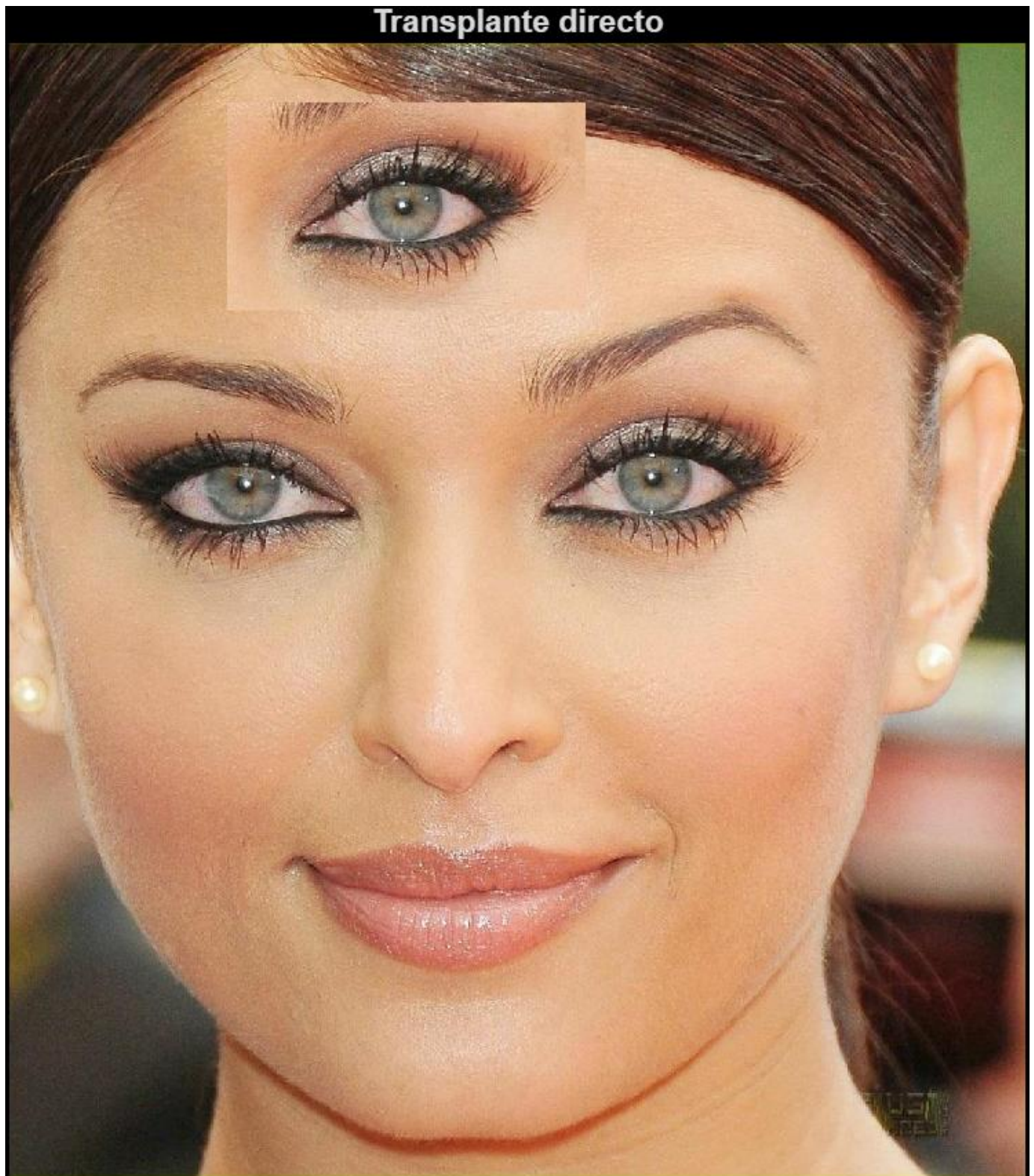
3.2 Aplicación: Fusión de imágenes

Dad las coordenadas X_{dest} , Y_{dest} que habéis elegido.

ydest = 175;

xdest = 425;

Adjuntad captura del resultado.

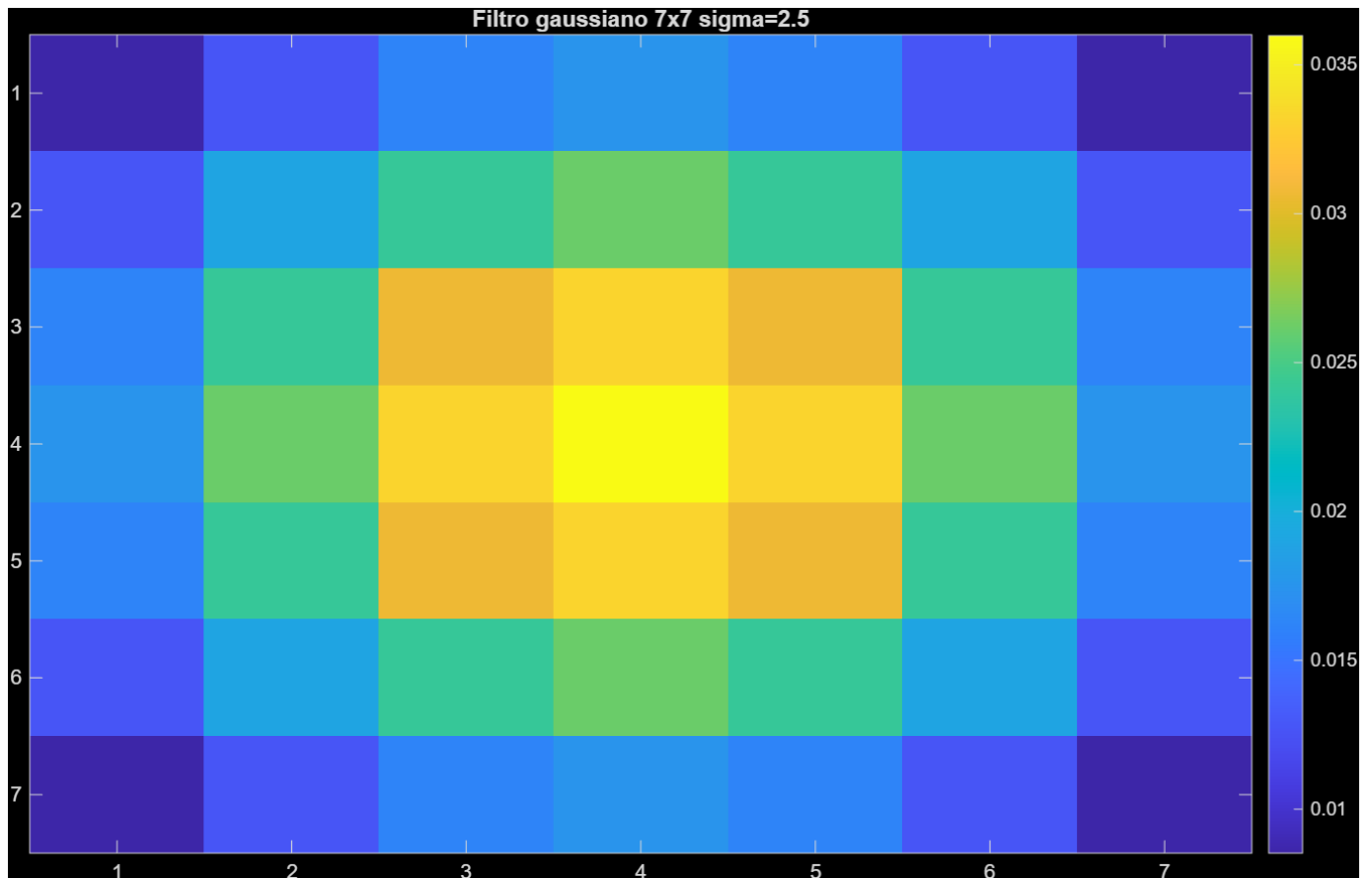


Adjuntad una captura del resultado usando la máscara

Fusion simple con mascara binaria



Volcad una copia del filtro gaussiano usado.



Adjuntar vuestro código para la creación de las pirámides, mezcla y recuperación de la imagen final. Adjuntad imagen del resultado usando esta técnica.

%Fusion imagenes

```
im = im2double(imread('face_img.jpg'));

% Definir zonas (ejemplo, ajusta con tu click)
ry=(-111:112); rx=(-191:192);
Yorg=466;
Xorg=692;
z1=im(Yorg+ry,Xorg+rx,:);

ydest = 175;
xdest = 425;

res1 = im;

res1(ydest+ry,xdest+rx,:)=z1;

figure;
imshow(res1);
title('Transplante directo');

z0 = im(ydest+ry-1,xdest+rx-1, :);
```

```
m = crea_mask(size(z0), [90 180]);

z_simple = m .* z1 + (1 - m) .* z0;
re2 = im;

re2(ydest+ry-1,xdest+rx-1, :) = z_simple;

figure;
imshow(re2);
title('Fusion simple con mascara binaria');

N = 5;

p0 = lap(z0, N);
p1 = lap(z1, N);

pm = cell(1,N);
pm{1} = m;

for k = 2:N
    g = fspecial('gaussian', [7 7], 2.5);

    temp = imfilter(pm{k-1}, g, 'replicate');
    temp = temp(1:2:end, 1:2:end, :);

    pm{k} = temp;
end

g = fspecial('gaussian', [7 7], 2.5);

figure;
imagesc(g);
colorbar;
title('Filtro gaussiano 7x7 sigma=2.5');

mix = cell(1,N);

for k = 1:N
    mix{k} = pm{k} .* p1{k} + (1 - pm{k}) .* p0{k};
end

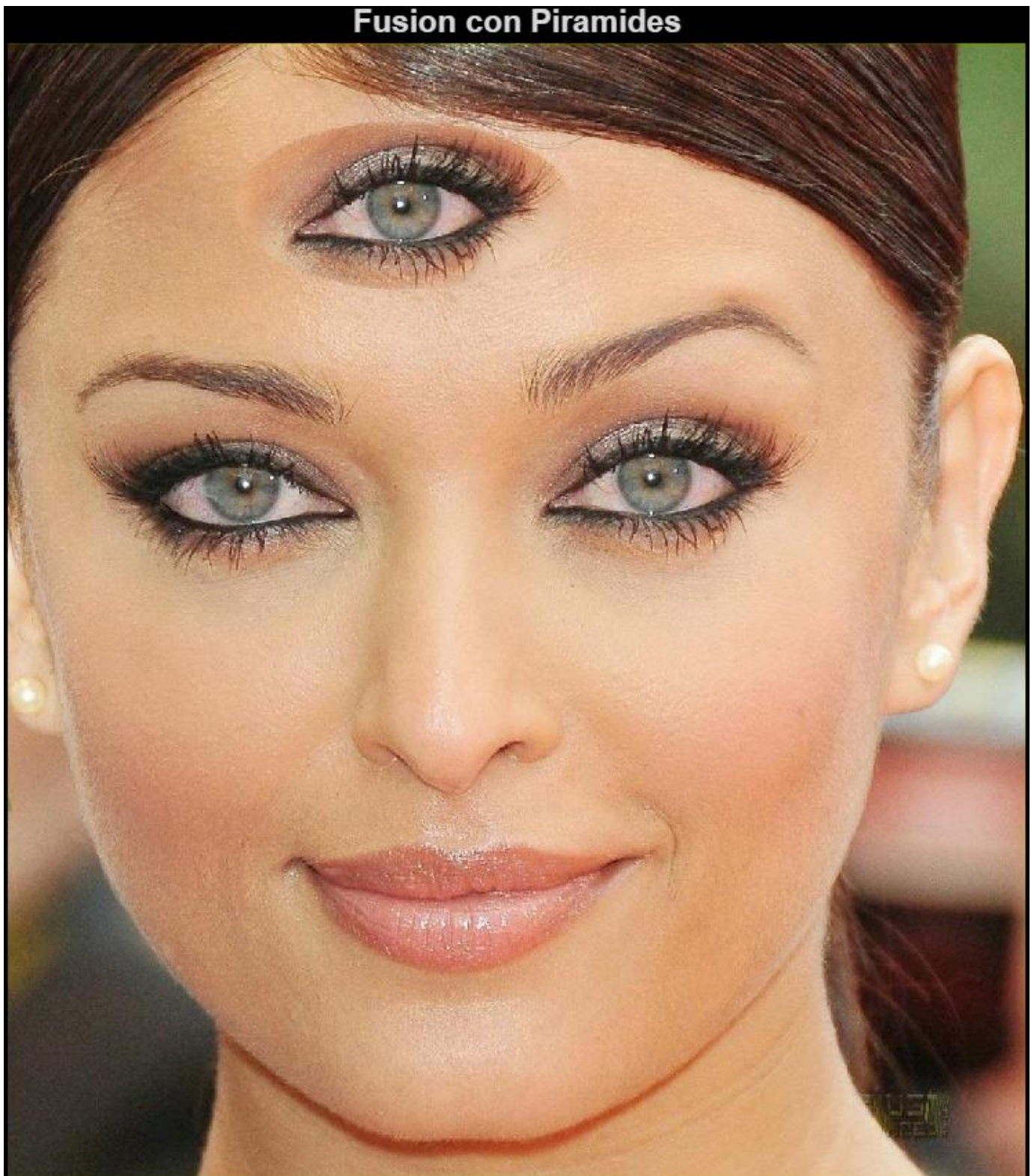
z_mix = inv_lap(mix);

res3 = im;
res3(ydest+ry-1,xdest+rx-1, :) = z_mix;

figure;
imshow(res3);
```



```
title('Fusion con Piramides');
```



Usando vuestras propias imágenes:

Adjuntad las fotos originales y la imagen final obtenida.



En la entrega, junto con el pdf/doc de la hoja de respuestas subid las dos fotos usadas (formato jpg) y el programa fusion.m que combine ambas fotos y obtenga el resultado final.

3.3 Aplicación: alineación de imágenes

Adjuntad una captura de la imagen resultado.



Alineación automática usando pirámides:

Adjuntad código de la función `registra()` una vez completada

`function [dx, dy] = registrar(im, ref)`

```
N = 5;  
r = 15;  
pyr_im = cell(1,N);  
pyr_ref = cell(1,N);  
  
pyr_im{1} = im;  
pyr_ref{1} = ref;  
  
for k = 2:N  
    pyr_im{k} = imresize(pyr_im{k-1}, 0.5);  
    pyr_ref{k} = imresize(pyr_ref{k-1}, 0.5);  
end
```

```

end

dx = 0;
dy = 0;

for k = N:-1:1

    im_k = pyr_im{k};
    ref_k = pyr_ref{k};

    dx = round(2*dx);
    dy = round(2*dy);

    best = -inf;
    best_dx = 0;
    best_dy = 0;

    for i = -r:r
        for j = -r:r

            im_shift = circshift(im_k, [dy+i, dx+j]);
            val = sum(sum(im_shift .* ref_k));

            if val > best
                best = val;
                best_dx = dx + j;
                best_dy = dy + i;
            end
        end
    end

    dx = best_dx;
    dy = best_dy;

    disp(['Nivel ' num2str(k) ' -> dx=' num2str(dx) ' dy=' num2str(dy)]);
end
end

```

Volcad los desplazamientos (dX,dY) que se van obteniendo en cada nivel entre el canal G y el B

```

Nivel 5 -> dx=0 dy=1
Nivel 4 -> dx=0 dy=3
Nivel 3 -> dx=0 dy=5
Nivel 2 -> dx=0 dy=10
Nivel 1 -> dx=1 dy=21
Nivel 5 -> dx=0 dy=3
Nivel 4 -> dx=0 dy=6
Nivel 3 -> dx=-1 dy=12

```


Nivel 2 -> dx=-1 dy=23

Nivel 1 -> dx=-3 dy=47

Volcad los desplazamientos finales del canal rojo R con respecto al canal azul B.

Final:

1 21 -3 47

Adjuntar código del script y la imagen color obtenida.

```
im = im2double(imread('multiplex.jpg'));
```

```
h = size(im,1)/3;
```

```
B = im(1:h,:);
```

```
G = im(h+1:2*h,:);
```

```
R = im(2*h+1:3*h,:);
```

```
Cx = 1792/2;
```

```
Cy = 1600/2;
```

```
rx = (-800:800);
```

```
ry = (-650:650);
```

```
b = B(Cy+ry, Cx+rx);
```

```
g = G(Cy+ry, Cx+rx);
```

```
r = R(Cy+ry, Cx+rx);
```

```
[dxg, dyg] = registrar(g, b);
```

```
[dxr, dyr] = registrar(r, b);
```

```
disp('Final:');
```

```
disp([dxg dyg dxr dyr]);
```

```
g_al = circshift(g, [dyg dxg]);
```

```
r_al = circshift(r, [dyr dxr]);
```

```
im_color = cat(3, r_al, g_al, b);
```

```
figure;
```

```
imshow(im_color);
```

```
title('Imagen alineada');
```



Adjuntad una captura de un recorte de la cara de más a la izquierda.



```
figure;  
imshow(im_color(370:620,50:270,:));  
title('Zoom cara izquierda');
```

Para la nueva imagen dad los desplazamientos dX, dY finales para los canales G y R.

G: $dx=5$ $dy=-9$

R: $dx=8$ $dy=4$

Adjuntad la imagen final resultante.

